

PBL 04: EC2 중심 관리에서 관리형 서비스로 클라우드 아키텍처 재설계 보고서

정하은

1. 개요

본 보고서는 EC2 인스턴스를 이용하여 NAT 게이트웨이 기능과 데이터베이스 서비스를 제공함으로써 발생하는 관리자의 운영 부담을 분석하고, 이를 AWS의 관리형 서비스로 전환하여 해결하는 클라우드 아키텍처 재설계 방안을 제안하고자 한다. AWS Well-Architected Framework의 원칙을 기반으로 관리 작업의 효율성을 극대화하고, 아키텍처의 확장성 및 유연성을 확보하는 데 중점을 두었다.

2. 문제 정의 및 요구사항

관리자는 현재 EC2 인스턴스 기반의 NAT 게이트웨이 및 MySQL 데이터베이스 운영으로 인해 패치, 백업, 고가용성 구성, 스케일링 등 각 인스턴스 관리 작업에 상당한 시간과 노력을 투입해야 하는 부담을 느끼고 있다. 이에 따라, 관리 작업을 줄이고 워크로드의 효율성을 높일 수 있도록 기존 아키텍처를 AWS 관리형 서비스를 이용하여 재설계할 필요가 있어 보인다.

기존 아키텍처 구성

- VPC 안에 Public Subnet과 Private Subnet 구역이 각각 존재
- Public Subnet 안에는 NAT Instance 위치
- Private Subnet 안에는 DB Server (MySQL) 위치

3. AWS Well-Architected Framework의 이해 및 적용 원칙

AWS Well-Architected Framework는 클라우드에서 안정적이고, 안전하며, 효율적이고, 비용 효율적인 시스템을 구축하기 위한 모범 사례를 제공한다. 본 재설계 과제에서는 특히 다음 원칙들을 중점적으로 고려했다.

- **운영 우수성(Operational Excellence)** : 클라우드에서 워크로드를 효과적으로 실행하고, 비즈니스 가치를 지속적으로 제공하며, 지원 프로세스를 개선 (자동화, 관리 오버헤드 감소)
- **안정성(Reliability)** : 인프라 또는 서비스 중단으로부터 복구하고, 수요 증가에 따라 동적으로 확장하며, 시스템 오류를 완화 (고가용성, 장애 조치)
- **성능 효율성(Performance Efficiency)** : 컴퓨팅 리소스를 효율적으로 사용하고, 시스템 요구 사항 변화에 맞춰 리소스를 최적화 (적절한 리소스 크기, 스케일링)
- **비용 최적화(Cost Optimization)** : 불필요한 비용을 제거하고, 투자 수익을 극대화 (관리 오버헤드 감소, 필요에 따른 확장)

4. 관리형 서비스와 비관리형 서비스의 차이점

두 서비스 유형의 차이를 명확히 파악하여 관리형 서비스로의 전환 필요성을 이해하고자 한다.

비관리형 서비스 (Unmanaged Service, 예: EC2)

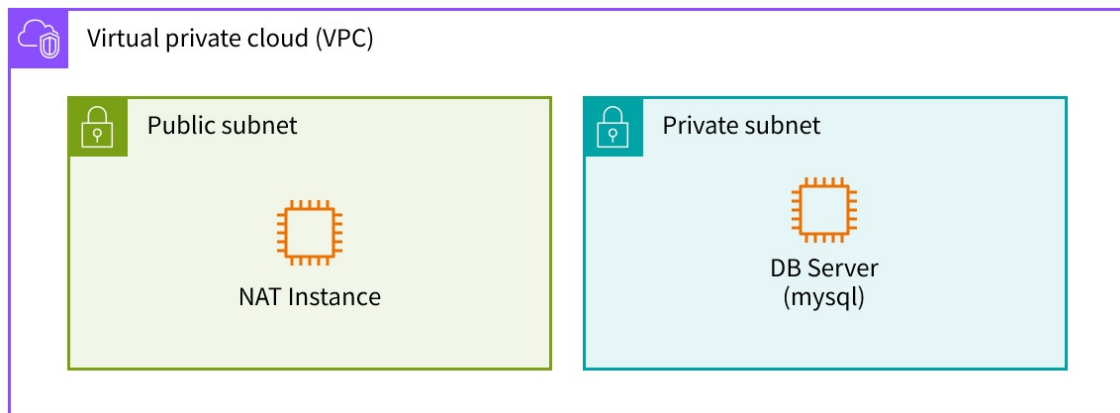
- **사용자 책임** : OS 패치, 애플리케이션 설치 및 구성, 백업, 고가용성, 스케일링, 모니터링 등 대부분의 인프라 및 소프트웨어 운영을 사용자가 직접 관리해야 한다.
- **장점** : 높은 수준의 유연성과 제어권을 제공한다.
- **단점** : 운영 및 관리 부담이 크고, 전문 지식과 인력 투입이 필요하며, 고가용성 및 확장성을 직접 설계하고 구현해야 한다.

관리형 서비스 (Managed Service, 예: Amazon RDS, AWS NAT Gateway)

- **AWS 책임** : AWS가 운영 체제, 미들웨어, 백업, 패치, 스케일링, 모니터링, 고가용성 구성 등 대부분의 인프라 관리 작업을 대신 수행한다.
- **장점** : 관리 부담이 감소하여 사용자는 핵심 비즈니스 로직에 집중할 수 있다. 높은 가용성, 확장성, 보안 기능이 기본으로 제공되며, 운영 비용(인력, 시간)을 절감할 수 있다.
- **단점** : AWS가 제공하는 옵션 내에서만 제어가 가능하여 유연성이 제한될 수 있다.

5. 기존 아키텍처 분석 및 관리 부담 원인 식별

주어진 아키텍처 구성도는 Public Subnet과 Private Subnet으로 나누어 보안을 강화하고, Private Subnet의 리소스가 외부 인터넷에 접근할 수 있도록 NAT Instance를 사용하는 전형적인 이중 계층(2-Tier) 또는 삼중 계층(3-Tier) 아키텍처의 백엔드 구성으로 파악된다.



이러한 구성에서 관리 부담을 야기하는 주요 원인은 다음과 같다.

NAT Instance (Public Subnet 내 EC2 인스턴스)

- **패치 및 보안** : NAT 역할을 하는 EC2 인스턴스의 운영 체제에 대한 정기적인 보안 패치 및 업데이트를 수동으로 수행해야 한다.
- **고가용성** : 단일 인스턴스이므로 단일 장애점(Single Point of Failure)이 될 수 있다. 고가용성을 확보하려면 Auto Scaling Group, Health Check, 다중 AZ 배포 등을 직접 구성하고 관리해야 한다.
- **성능 및 스케일링** : 트래픽 증가에 따라 인스턴스 유형을 수동으로 변경하거나(Scale Up), 여러 NAT 인스턴스를 관리해야 하는(Scale Out) 부담이 있다.

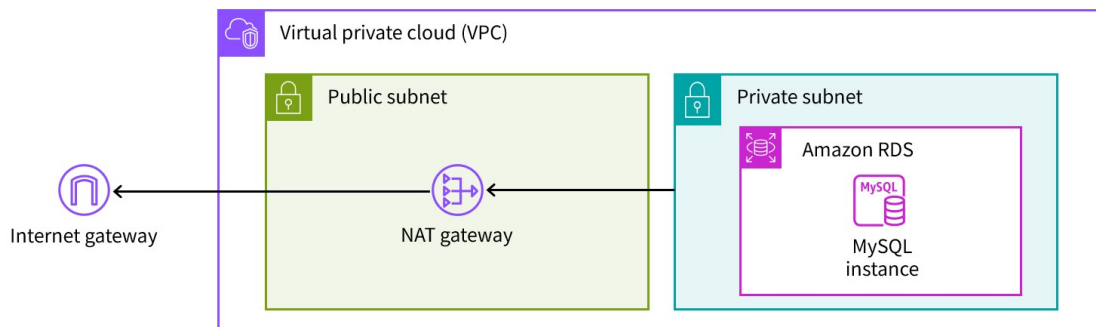
DB Server (MySQL on EC2, Private Subnet 내 EC2 인스턴스)

- **데이터베이스 엔진 관리** : MySQL 설치, 구성, 튜닝, 버전 업그레이드 등 DB 엔진 자체에 대한 전문적인 관리 지식이 필요하다.
- **백업 및 복구** : 데이터 유실 방지를 위한 백업 전략 수립(주기, 보존 기간) 및 실행, 그리고 장애 발생 시 복구 절차를 직접 관리해야 한다.
- **고가용성** : DB 서버의 장애는 서비스 전체에 치명적이므로, Multi-AZ 복제, 페일오버 등 고가용성 구성을 직접 설계하고 구현해야 한다.
- **성능 모니터링 및 최적화** : DB 성능 저하 시 원인 분석 및 쿼리 최적화 등 복잡한 작업이 필요하다.
- **보안** : OS 및 DB 소프트웨어의 보안 취약점을 지속적으로 관리해야 한다.

이처럼, 기존 아키텍처는 인프라 및 소프트웨어 운영에 필요한 광범위한 수동 작업으로 인해 관리자의 시간과 노력을 과도하게 소모하는 문제가 있다.

6. 관리형 서비스 기반 재설계

기존 아키텍처의 핵심 기능(NAT 기능, MySQL DB 기능)을 유지하되, Well-Architected Framework의 원칙에 따라 관리 부담을 줄이고 안정성 및 확장성을 높이는 방향으로 재설계했다. 이는 기존 EC2 기반 구성 요소를 AWS의 적절한 관리형 서비스로 "대체"하는 방식으로 이루어졌다.



6.1. NAT Instance → AWS NAT Gateway 전환

- **전환 이유** : NAT Instance의 패치, 고가용성, 스케일링 등 관리 부담을 해소하고, 더 안정적이고 고성능의 서비스를 이용하기 위해서이다.
- **AWS NAT Gateway의 장점**
 - **운영 우수성** : AWS가 관리하는 서비스이므로, 사용자는 인스턴스 관리, 패치, OS 업데이트 등에 신경 쓸 필요가 없다.
 - **안정성** : 각 가용 영역(AZ) 내에서 높은 가용성을 제공하며, AWS가 자동으로 장애 조치 및 스케일링을 관리한다. 단일 장애점이 제거된다.
 - **성능 효율성** : 높은 대역폭과 트래픽 처리량을 제공하여 트래픽 증가에 자동으로 대응한다.
 - **비용 최적화** : 사용량에 따라 비용이 청구되어, 인스턴스 유지보수 및 인력 비용 측면에서 총 소유 비용(TCO)이 절감된다.
- **아키텍처 변경** : Public Subnet에 NAT Gateway를 생성하고, Private Subnet의 라우팅 테이블에서 인터넷으로 향하는 모든 트래픽('0.0.0.0/0')의 다음 홉(Next Hop)을 새로 생성된 NAT Gateway로 지정하도록 한다.

6.2. DB Server (MySQL on EC2) → Amazon RDS for MySQL 전환

- **전환 이유** : EC2 상에서 직접 MySQL을 운영할 때 발생하는 DB 엔진 및 OS 패치, 백업, 고가용성 구성, 스케일링 등 복잡하고 시간 소모적인 관리 작업을 AWS에 이관하기 위해서이다.
- **Amazon RDS for MySQL의 장점**
 - **운영 우수성** : AWS가 데이터베이스 엔진 및 운영 체제의 패치, 백업, 모니터링 등 대부분의 관리 작업을 자동화하여 제공하므로, 관리자는 애플리케이션 개발 및 데이터 관리에 집중할 수 있다.
 - **안정성** : Multi-AZ 배포 옵션을 통해 자동으로 다른 가용 영역에 대기 인스턴스를 프로비저닝하고 동기 복제를 수행한다. 프라이머리 인스턴스 장애 시 자동으로 대기 인스턴스로 페일오버되어 애플리케이션 가동 중단 시간을 최소화한다.
 - **성능 효율성** : 클릭 몇 번으로 컴퓨팅 및 스토리지 리소스를 쉽게 스케일 업/다운할 수 있어, 변화하는 워크로드 요구사항에 유연하게 대응할 수 있다.
 - **비용 최적화** : 관리 오버헤드 감소 및 필요한 만큼만 리소스를 사용하는 종량제 과금 방식을 통해 비용 효율성을 높인다.
 - **보안** : VPC 내 Private Subnet 배포, 암호화 기능(저장 중/전송 중 데이터) 등을 통해 데이터베이스 보안을 강화할 수 있다.
- **아키텍처 변경** : Private Subnet 내에 Amazon RDS for MySQL 인스턴스를 생성하고 고가용성을 위해 일반적으로 최소 두 개 이상의 Private Subnet에 Multi-AZ 배포를 구성하는 것이 권장된다.

7. 결론

본 보고서에서는 기존 EC2 중심의 관리형 아키텍처가 가진 운영 부담을 AWS 관리형 서비스인 NAT Gateway와 Amazon RDS for MySQL로 대체하도록 제안하였다. 이러한 전환을 통해 다음과 같은 이점을 얻을 수 있을 것이다.

- **관리 시간 및 노력 절감** : 인프라 관리의 대부분을 AWS에 위임하여, 관리자는 핵심 애플리케이션 개발 및 비즈니스 성장에 집중할 수 있다.
- **향상된 안정성 및 가용성** : NAT Gateway의 자동 장애 조치 및 RDS Multi-AZ 배포를 통해 서비스 중단 위험을 최소화하고, 재해 복구 능력을 강화한다.
- **유연한 확장성** : 수요 변화에 따라 리소스를 쉽고 빠르게 확장하거나 축소할 수 있다.
- **비용 효율성** : 불필요한 관리 비용을 줄이고, 사용량 기반의 과금 모델을 통해 리소스를 최적화할 수 있다.

결과적으로 본 과제에서는 AWS Well-Architected Framework의 원칙을 적용하여, 효율적이고 확장 가능한 클라우드 아키텍처를 재설계하는 사례를 제시하였다.